МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №4
по курсу «Программирование графических процессоров»

Работа с матрицами. Метод Гаусса. Вычисление детерминанта матрицы (вариант 1)

Выполнил: Голев Андрей Михайлович
Группа: М8О-401Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы — использование объединения запросов к глобальной памяти: реализация метода Гаусса с выбором главного элемента по столбцу, знакомство с библиотекой алгоритмов для параллельных расчётов Thrust (использовать её разрешено только для поиска максимального элемента на каждой итерации алгоритма), работа с двухмерной сеткой потоков и исследование производительности программы с помощью утилиты nvprof.

Мой вариант — №1, вычисление детерминанта матрицы. На вход даётся число n — размер матрицы, затем n строк по n вещественных чисел — элементы матрицы; $n \le 10^4$. Вывод — одно число, детерминант матрицы. Вещественный тип данных — double; при сравнении по модулю с нулём используется порог 10^{-7}; результат выводится с относительной точностью 10^{-10}. Пример из условия: для матрицы 3×3 с элементами 1 2 3 / 4 5 6 / 7 8 7 ответ — 6.0000000000e+00.

Программное и аппаратное обеспечение

Код я писал, отлаживал и компилировал на виртуальной машине с GPU, на ней же проводил все замеры времени и профилировку.

Графический процессор — NVIDIA GeForce GTX 1650: compute capability 7.5, 14 мультипроцессоров, глобальная память 4 086 169 600 байт (примерно 3,8 ГБ), разделяемая память на блок 48 КБ, константная память 64 КБ, 65 536 регистров на блок, максимум 1024 нити на блок, максимальная сетка (2 147 483 647, 65 535, 65 535), размер варпа 32.

Из софта: Ubuntu 24.04.1 LTS (ядро 6.8.0-84-generic), драйвер NVIDIA 550.144.03, CUDA 12.4, компилятор nvcc 12.0 (V12.0.140), g++ 13.3.0. Боевая программа компилируется командой из условия: nvcc -Werror cross-execution-space-call -lm. Для замеров я написал отдельный бенчмарк bench4.cu (компилируется тем же nvcc с -O2, CPU-эталон в нём — на OpenMP), генератор входных данных gen_input.py и скрипт run_bench.sh, который компилирует всё, снимает параметры GPU, прогоняет nvprof на боевой программе и бенчмарк и собирает таблицы для отчёта.

Метод решения

Детерминант считается методом Гаусса с выбором главного элемента по столбцу. На итерации k в текущем столбце k ищется максимальный по модулю элемент; его строка меняется местами со строкой k (при этом детерминант меняет знак); ведущий элемент умножается в накопитель det; затем из каждой строки ниже k вычитается строка k с множителем f[i] = A[i][k]/pivot — текущий столбец обнуляется. После n итераций матрица приведена к верхнетреугольному виду, и детерминант равен ±произведению ведущих элементов. Если $|pivot| < 10^{-7}$, матрица считается вырожденной и детерминант — ноль. Выбор главного элемента нужен для численной устойчивости: без него маленькие ведущие элементы приводят к быстрому росту погрешности округления.

Чтобы обращения к глобальной памяти объединялись (coalescing), матрица хранится по столбцам: элемент A[i][j] лежит по адресу $j \cdot n + i$, то есть столбец матрицы — непрерывный кусок памяти. Тогда все горячие доступы — поиск максимума по хвосту столбца k, перестановка строк k и pr и внутренний цикл исключения по строкам i при фиксированном столбце j — идут по непрерывной памяти, и соседние нити варпа обращаются к соседним элементам.

Итерации алгоритма последовательны (каждая зависит от результата предыдущей), поэтому цикл по k ведётся на хосте. На каждой итерации: thrust::max_element(thrust::device, …, AbsLess()) ищет максимальный по модулю элемент хвоста столбца k — это единственное разрешённое использование Thrust; ведущий элемент копируется на хост (он нужен для накопителя det и проверки на ноль); если главный элемент не на диагонали — запускается ядро перестановки строк; затем ядро множителей и двумерное ядро исключения пересчитывают подматрицу. Все ядра используют grid-stride циклы с фиксированной конфигурацией запуска, так что число нитей не зависит от n.

Ввод: на предельном $n = 10^4$ это до 10^8 чисел, поэтому написан собственный буферизованный парсер чисел с плавающей точкой (знак, целая часть, дробная часть, экспонента) поверх fread — стандартный scanf на таком объёме слишком медленный. Вывод — printf с форматом %.10e, десять знаков после точки, что соответствует требуемой относительной точности 10^{-10}.

Описание программы

Программа — один файл lab4.cu.

Три ядра. swapRowsKernel(a, n, k, pr) переставляет строки k и pr: нити grid-stride циклом идут по столбцам j и меняют местами A[k][j] и A[pr][j]. factorKernel(a, f, n, k, pivot) считает множители f[i] = A[i][k]/pivot для строк ниже k, читая столбец k — непрерывный кусок памяти. eliminateKernel(a, f, n, k) — главное ядро, двумерный grid-stride цикл: координата y нити идёт по столбцам j подматрицы, координата x — по строкам i, и каждая нить вычисляет A[i][j] −= f[i]·A[k][j]; при фиксированном j последовательные i — это последовательные адреса, поэтому обращения соседних нитей варпа объединяются в одну транзакцию, а не меняющийся внутри цикла элемент A[k][j] читается из кэша. Нулевые множители пропускаются (проверка f[i] != 0).

Функтор AbsLess — сравнение по модулю |a| < |b|, помечен __host__ __device__, чтобы работать в thrust::max_element на устройстве.

readDouble — буферизованный парсер double из stdin (буфер 1 МБ): пропускает пробельные символы, читает знак, целую часть, дробную часть (до 17 разрядов с накоплением в unsigned long long и делением на степень десяти) и экспоненту; при исчерпании буфера подчитывает данные fread'ом.

fail(msg) печатает «ERROR: …» в stderr и завершается с кодом 0 — по требованию курса программа не должна завершаться аварийно ни при каких условиях.

main читает n (с проверкой $0 \le n \le 10^4$), читает матрицу, сразу укладывая её по столбцам, выделяет память на хосте (malloc) и устройстве (cudaMalloc: матрица n·n double и вектор множителей f длины n), копирует матрицу на устройство и ведёт цикл исключения. Каждый вызов CUDA проверяется, при ошибке — fail. Результат — det·sign — печатается с десятью знаками, память освобождается. Случай n = 0 — матрица вырождается в единичную, ответ 1.

Рабочая конфигурация запуска eliminateKernel — grid(16×16), block(32×8): 256 блоков по 256 нитей, всего 65 536 нитей; перестановка строк и множители запускаются одномерной конфигурацией <<<128, 256>>> (32 768 нитей). Все конфигурации фиксированы и не зависят от размера входа; остальные конфигурации из таблицы ниже использовались только для замеров.

Результаты

Время мерял событиями CUDA (cudaEventRecord/cudaEventElapsedTime) в бенчмарке bench4.cu: интервал охватывает весь цикл исключения — от первого поиска максимума до последнего ядра. В таблицах — среднее по 10 запускам для n = 256 и по 5 запускам для n = 1024, перед замерами — прогревочные запуски (3 и 2 соответственно). Матрицы генерируются детерминированным конгруэнтным ГПСЧ (seed 42): элементы равномерны на $\pm\sqrt{(12 \cdot e/n)}$, при такой дисперсии детерминант не переполняет double. Перебор конфигураций выполнен для n = 256 и n = 1024; замеры для больших размеров (3000 и 4096) не были завершены в отведённое время, поэтому большие матрицы представлены профилировкой nvprof боевой программы. Результат каждой конфигурации сверялся с CPU-эталоном (тот же алгоритм с выбором главного элемента, OpenMP, 2 потока): всего 28 проверок, максимальное относительное расхождение $3{,}6 \cdot 10^{-13}$.

Таблица 1 — Время цикла исключения по конфигурациям

Конфигурация (сетка; блок)	Всего нитей	n=256, мс	n=1024, мс
<<<1, 32>>> (1×1; 32×1)	32	1644,057	13889,104
<<<1, 64>>> (1×1; 64×1)	64	1544,082	9115,306
<<<1, 128>>> (1×1; 128×1)	128	1062,416	6353,011
<<<1, 256>>> (1×1; 256×1)	256	1642,876	7393,496
<<<1, 512>>> (1×1; 512×1)	512	1630,368	5411,050
<<<4, 256>>> (2×2; 16×16)	1 024	1600,608	5847,148
<<<16, 256>>> (4×4; 16×16)	4 096	1566,484	5595,480
<<<64, 256>>> (8×8; 16×16)	16 384	1506,324	4879,235
<<<256, 256>>> (16×16; 16×16)	65 536	1194,679	4481,426
<<<1024, 256>>> (32×32; 16×16)	262 144	1632,164	4242,404
<<<256, 256>>> (16×16; 32×8)	65 536	1620,219	4003,998
<<<1024, 256>>> (32×32; 32×8)	262 144	1599,858	3598,726
<<<1024, 1024>>> (32×32; 32×32)	1 048 576	1573,797	3758,220
CPU (OpenMP, 2 потока)	2 потока	442,804	2805,558

На n = 256 время почти не зависит от конфигурации (1,06–1,64 с): подматрицы маленькие (в среднем порядка 85×85 элементов), и время определяют не вычисления, а 256 итераций накладных расходов — на каждой поиск максимума через Thrust (несколько ядер и синхронизация), копирование ведущего элемента на хост и два-три запуска ядер. Лучший результат здесь у <<<1, 128>>> (1062,416 мс): одному блоку хватает работы, а лишние блоки только добавляют расходов на запуск.

На n = 1024 вычисления уже заметнее. Конфигурации с одним блоком (<<<1, 32>>>…<<<1, 512>>>) работают на одном мультипроцессоре из четырнадцати и заметно хуже — от 5,4 до 13,9 с. С ростом числа блоков время падает: 4,88 с у <<<64, 256>>>, 4,48 с у <<<256, 256>>> (16×16; 16×16) и минимум 3,60 с у <<<1024, 256>>> (32×32; 32×8). Рабочая конфигурация программы — <<<256, 256>>> (16×16; 32×8), 4,00 с, всего на 11% хуже лучшей; вытянутый блок 32×8 оказался удачнее квадратного 16×16 при том же числе нитей (4,00 с против 4,48 с): 32 нити по x — целый варп на строку подматрицы.

CPU-эталон на OpenMP (2 потока) — 442,804 мс на n = 256 и 2805,558 мс на n = 1024. GPU проигрывает CPU на обоих наборах: в 2,4 раза на маленьком и в 1,3 раза на большом (лучшая конфигурация 3598,726 мс против 2805,558 мс). Причина — не скорость

вычислений, а устройство алгоритма: итерации последовательны, каждая требует результата предыдущей, и на каждой из n итераций происходит синхронизация хост–устройство; GPU большую часть времени простаивает (это видно по nvprof ниже).

Влияние объединения запросов мерял на рабочей конфигурации: в uncoalesced-варианте измерения x и y в eliminateKernel переставлены местами, внутренний цикл идёт по столбцам j при фиксированной строке i — соседние нити варпа обращаются к элементам с шагом n·8 байт, и каждое обращение — отдельная транзакция.

Таблица 2 — Влияние объединения запросов к глобальной памяти (рабочая конфигурация)

Размер	Coalesced, мс	Uncoalesced (x↔y), мс	Отношение
n=256	1620,219	1513,936	0,93
n=1024	4003,998	4175,114	1,04

На n = 1024 uncoalesced-вариант медленнее на 4% (4175,114 против 4003,998 мс), на n = 256 разница в пределах шума измерений (1513,936 против 1620,219 мс). Эффект скромный, потому что ядро исключения занимает лишь малую долю общего времени цикла: объединение запросов ускоряет работу с памятью, но общее время определяют накладные расходы и синхронизации.

Профилировка боевой программы утилитой nvprof (входы с тем же генератором): n = 1024 и n = 3000. Thrust для разных длин столбца запускает разное число внутренних ядер, поэтому в таблицах они объединены в строку «ядра Thrust».

Таблица 3 — nvprof: разбивка по операциям GPU, боевая программа, n=1024

Операция GPU	Время, мс	Вызовов	Доля, %
Исключение (eliminateKernel)	57,293	1023	74,1
Копирование матрицы HtoD	5,621	1	7,3
Ядра Thrust (поиск максимума)	5,816	3072	7,5
Копирования DtoH (pivot, Thrust)	3,154	2048	4,1
Перестановка строк (swapRowsKernel)	3,051	1016	4,0
Множители (factorKernel)	2,359	1023	3,0
Вся GPU-активность	77,294	—	100

Таблица 4 — nvprof: разбивка по операциям GPU, боевая программа, n=3000

Операция GPU	Время, мс	Вызовов	Доля, %
Исключение (eliminateKernel)	1452,560	2999	92,5
Копирование матрицы HtoD	56,215	1	3,6
Ядра Thrust (поиск максимума)	21,737	10 720	1,4
Перестановка строк (swapRowsKernel)	20,338	2991	1,3
Копирования DtoH (pivot, Thrust)	9,107	6000	0,6
Множители (factorKernel)	7,571	2999	0,5
Вся GPU-активность	1567,528	—	100

Картина по nvprof такая. На n = 1024 вся GPU-активность — 77,294 мс, то есть около 2% от 4003,998 мс замеренного времени цикла исключения; остальные ~98% времени GPU простаивает в ожидании хоста. По API-статистике cudaStreamSynchronize заняла 6,33 с на 4096 вызовах — по четыре синхронизации на итерацию. Само ядро

исключения — 74,1% GPU-времени (57,293 мс на 1023 вызова), остальное — служебные операции: копирование матрицы на устройство (7,3%), ядра Thrust и его копирования (7,5% и 4,1%), перестановка строк (4,0%) и множители (3,0%). На n = 3000 вычисления перевешивают: eliminateKernel занимает 92,5% GPU-времени (1452,560 мс из 1567,528 мс), а доля служебных операций падает до ~7,5%. С ростом n доля полезной работы растёт, и на больших матрицах баланс должен смещаться в пользу GPU.

Сквозная проверка боевой программы под nvprof: детерминант на n = 1024 — $-2.0156224448 \cdot 10$ против $-2.015622444839078 \cdot 10$ у CPU-эталона бенчмарка, относительное расхождение $1{,}9 \cdot 10^{-11}$; на n = 3000 — 1.7747208149 (CPU-эталон для этого размера не был завершён).

Выводы

Детерминант — одна из базовых характеристик квадратной матрицы: он отличает вырожденные матрицы от невырожденных, через него считаются ранг, обратная матрица и решения СЛАУ (правило Крамера), а в аналитической геометрии он задаёт ориентированный объём. Метод Гаусса с выбором главного элемента по столбцу — стандартный численно устойчивый способ его вычисления за $O(n^3)$ операций.

Главный урок этой работы: метод Гаусса принципиально последователен по итерациям исключения — каждая итерация зависит от результата предыдущей, поэтому на GPU переносится только внутренность итерации (поиск максимума и пересчёт подматрицы), а каждая итерация требует синхронизации хост–устройство. На матрицах n = 256 и n = 1024 накладные расходы доминируют: по nvprof GPU-активность — лишь ~2% времени цикла (77 мс из 4004 мс на n = 1024), и GPU проигрывает CPU-эталону на OpenMP (3,60 с против 2,81 с на n = 1024). С ростом n доля полезных вычислений растёт (92,5% GPU-времени уже на n = 3000), так что на больших матрицах баланс меняется в пользу GPU.

Объединение запросов к глобальной памяти достигается хранением матрицы по столбцам: внутренний цикл исключения идёт по непрерывной памяти, и соседние нити варпа попадают в одну транзакцию. Измеренный эффект — 4% на n = 1024: скромно, потому что ядро исключения занимает малую долю общего времени, а сама коалесцированная версия ядра упирается не в пропускную способность памяти. Thrust удобен для редукций, но каждый вызов thrust::max_element с политикой thrust::device — это несколько ядер и синхронизация, что в цикле из n итераций заметно.

Возможные оптимизации: убрать копирование ведущего элемента на хост на каждой итерации (накапливать произведение и знак на устройстве, копировать пакетно), заменить thrust::max_element собственным ядром редукции (меньше ядер и копирований на итерацию), использовать CUDA Graphs, чтобы убрать расходы на повторные запуски ядер, и блочные (tile) версии исключения с разделяемой памятью для больших подматриц.